Q-Chain: Project Plan, Features, Roadmap and Team Guide

An AI-assisted quantum-safe migration assessment and verification platform. Built for the Security (incl. Blockchain & Cryptography) track by a team of five.

1. Project Overview

Most blockchain applications sign transactions with classical elliptic-curve signatures (ECDSA, secp256k1). A sufficiently capable quantum computer running Shor's algorithm could forge them. Migrating to post-quantum cryptography (PQC) is hard: developers must find the vulnerable code, choose an algorithm, confirm signatures still verify, and understand the size and speed cost.

Q-Chain is a web product that does this assessment for them. A developer points it at a sample blockchain app. Agents scan the code, flag classical signing, test NIST-standard PQC signatures against an ECDSA baseline, verify correctness and tamper-resistance, and produce a report. The hash of that report is anchored on a blockchain so anyone can verify later that it was not altered. Everything runs locally, so source code never leaves the developer's machine.

Honest scope statement (keep this in every pitch): Q-Chain assesses and verifies migration readiness. It does not claim that off-chain PQC signatures make existing on-chain transaction authorization quantum-safe, and the anchoring transaction itself still relies on the host chain's classical signatures.

2. How Each Hackathon Requirement Is Satisfied

Expectation	How Q-Chain satisfies it	Proof to show judges
Track: Security (Blockchain & Cryptography)	PQC migration assessment plus blockchain-anchored report integrity	Live scan, PQC benchmark, on-chain hash verification

Expectation	How Q-Chain satisfies it	Proof to show judges
Agentic browser or system use	Agents act on the machine: read the repo, run liboqs tests, execute benchmarks, call the chain. The web UI is driven by agent events in real time	Live agent activity feed while a scan runs
Ultra-low-latency voice	Mic button in the web app. Streaming speech-to-text, a fast intent router, spoken summary of results	Latency counter on screen (speech end to first spoken word)
System One decision models	A lightweight triage model ranks findings in milliseconds. Deterministic scoring rules, optionally backed by a small local classifier	Decision latency shown per scan (target under 50 ms)
Edge models	Small LLM and speech models run locally (Ollama, Whisper, Piper). No code is sent to cloud APIs. The result is delivered as a polished, deployable web product	Airplane-mode demo; "Local models only" badge in the UI
Multi-agent orchestration	Analysis, PQC testing, verification and reporting agents coordinated by an orchestrator	Agent graph view in the dashboard
MCP	Four MCP servers expose code analysis, crypto testing, benchmarking and blockchain tools to the agents with allow-listed, logged access	MCP call log panel

Open item: the brief mentions "System One Decision Models (e.g., Jev)". Ask the organizers what Jev refers to. If it is a specific model or tool, swap it into the decision layer and keep our rule engine as the fallback.

Open item: "Edge models" is read here both as on-device models and as a final-product-quality web experience. This plan covers both readings.

3. Scope Strategy

Building seven capabilities shallowly is the biggest risk. Build in this order, and do not start a tier until the one above it works end to end.

Tier 1: Core (must work flawlessly)

- Upload or select a sample app, scan, view severity-ranked findings
- PQC tests: ECDSA baseline vs ML-DSA vs SLH-DSA vs Falcon
- Verification: valid signatures pass, tampered messages fail
- Report generation and hash anchoring, plus a verification page
- Clean dashboard UI deployed at a live URL

Tier 2: Requirement features (needed to hit the brief)

- Multi-agent orchestration with a visible activity feed
- MCP servers behind the agents
- Local LLM (Ollama) for explanations
- Decision engine with measured latency
- Voice commands with measured latency

Tier 3: Stretch

- Migration patch suggestions (diff view)
- Hybrid signature demo (ECDSA + ML-DSA together)
- Report signed with ML-DSA before anchoring
- GitHub repo URL scanning
- Multi-language support beyond JavaScript and Solidity

4. Feature Specification

4.1 Code Scanner (Cryptographic Analysis Agent)

- Accepts a zip upload, a folder path, or one of three bundled sample apps (a token wallet, a voting dApp, a multisig).

- Detects classical crypto usage with rule-based patterns: `secp256k1`, `ECDSA`, `ecrecover`, `signMessage`, `ethers.Wallet`, `web3.eth.account`, `bitcoinjs ECPair`, `RSA`, `ed25519`, plus hash-and-sign flows.
- Each finding has: file, line, algorithm, context (key generation, signing, verification), quantum risk, and a confidence score.
- The local LLM explains each finding in plain language. The detection itself is deterministic so results are repeatable and defensible.

4.2 PQC Testing (PQC Testing Agent)

- Uses **liboqs** (via `liboqs-python`) for ML-DSA (FIPS 204), SLH-DSA (FIPS 205) and Falcon-512/1024 (FN-DSA is still being finalized, so label it Falcon).
- ECDSA secp256k1 baseline via the `cryptography` or `ecdsa` library.
- For each algorithm: key generation, sign, verify, over several message sizes, repeated enough times for stable averages.
- Confirm exact algorithm names against the installed liboqs version at start-up, since names vary between releases.

4.3 Verification (Verification Agent)

- Valid signature must verify.
- Flipped bit in the message must fail.
- Flipped bit in the signature must fail.
- Wrong public key must fail.
- Output a pass/fail matrix and a tamper-detection rate per algorithm.
- Metrics collected: public key size, signature size, keygen time, sign time, verify time, and the estimated on-chain storage or gas implication of the larger sizes.

4.4 Decision Engine (System One Layer)

- Input: findings plus benchmark results. Output: ranked findings and a recommended algorithm per use case.
- Scoring formula, for example: `risk = algorithm_weight x exposure x signature_lifetime x confidence`.
- Recommendation rules, for example: constrained gas or storage favors Falcon; conservative long-term security favors SLH-DSA; balanced general use favors ML-DSA.

- Runs in-process in milliseconds and displays its own latency. The LLM only writes the explanation text and never changes the ranking.
- Optional upgrade: a tiny scikit-learn classifier trained on rule-labelled synthetic findings, to justify the "decision model" wording.

4.5 Report and Blockchain Anchoring

- Report is a canonical JSON document (sorted keys) containing findings, benchmarks, verification matrix, tool versions and timestamp.
- Compute SHA-256 (or keccak256) of the canonical JSON.
- A small smart contract `ReportRegistry` stores `hash -> (submitter, timestamp)` and emits an event.
- Use a local Anvil/Hardhat chain for reliable demos, with an optional Sepolia deployment for credibility.
- **Verification page:** upload a report, recompute its hash, look it up on-chain, show Match or Tampered. Include a button that edits one character of the report to show the failure live.
- Stretch: sign the report with ML-DSA and include the public key and signature in the report before hashing.

4.6 Voice Interface

- Mic button in the dashboard header, push-to-talk.
- Speech-to-text: streaming local Whisper (faster-whisper, small or base model) or the browser Web Speech API as a fallback.
- A tiny intent router (keyword and pattern matching first, LLM only if no match) maps speech to actions: scan, compare, explain, anchor, verify.
- Text-to-speech: Piper locally, with the browser speech synthesis as fallback.
- Show a latency badge: end of speech to start of reply audio. Target under one second for simple commands.
- Example commands: "Scan the sample wallet", "Compare ML-DSA with ECDSA", "What is the highest risk finding?", "Anchor this report".

4.7 Web Dashboard (the "final product" layer)

- **Home / New Scan:** pick sample app or upload, privacy badge ("Runs locally").
- **Findings:** severity-ranked table, code snippet viewer, plain-language explanation.

- **Benchmarks:** charts for signature size, public key size, sign and verify latency across algorithms.
- **Agents:** live activity feed and agent graph, plus the MCP call log.
- **Report:** download JSON and PDF, anchor button, transaction link.
- **Verify:** drag-and-drop integrity check.
- Loading, empty and error states on every page, consistent design system, responsive layout, short onboarding.

5. System Architecture

```
[ Web App (React/Next.js) ]  <--WebSocket/REST-->  [ Backend API (FastAPI) ]
        |  mic audio                                        |
        v                                                   v
[ Voice Service: Whisper + Piper ]                [ Orchestrator ]
                                                    |   |   |   |
                           +------------------------+   |   |   +-------------
---+
                           v                            v   v
v
                 [ Analysis Agent ]        [ PQC Test Agent ] [ Verification
Agent ] [ Report Agent ]
                           |                            |             |
|
                           +-------------- MCP client --------------+--------
------------+
                                                 |
        +--------------------+--------------------+--------------------+
        v                     v                    v                    v
 [MCP: code-analysis] [MCP: crypto-test] [MCP: benchmark]     [MCP:
blockchain]
   (Semgrep/regex)      (liboqs + ECDSA)    (timing harness)   (web3 +
contract)
                                                 |
                              [ Decision Engine (in-process) ]
                              [ Ollama local LLM for explanations ]
```

Design rules

1. Deterministic tools do the work. The LLM explains and orchestrates but never decides security outcomes.

2. Every tool call goes through MCP and is logged and shown in the UI.
3. Agents communicate through a shared typed JSON schema (defined on day one, see section 8).
4. Nothing leaves the machine except the hash sent to the chain.

6. Technology Stack

Layer	Choice	Why
Frontend	Next.js or React + Tailwind + Recharts	Fast to build, good charts
Backend	Python FastAPI + WebSockets	Same language as liboqs and agents
Orchestration	Simple custom orchestrator, or LangGraph	Keep it light and debuggable
MCP	Official Python MCP SDK (four small servers)	Meets requirement cleanly
Local LLM	Ollama with a 3B to 8B instruct model (e.g., Llama 3.x or Qwen 2.5)	Edge model, runs on a laptop
PQC	liboqs + liboqs-python	Reference NIST implementations
Code analysis	Semgrep rules + regex + tree-sitter	Deterministic detection
Voice	faster-whisper, Piper, Web Speech API fallback	Low-latency local voice
Blockchain	Solidity, Foundry/Hardhat, Anvil, web3.py or ethers	Standard tooling
Hosting	Docker Compose locally; frontend on Vercel or similar for the live URL	Reproducible demo

Install risk: liboqs sometimes builds slowly. Prepare a Docker image on day one with liboqs and liboqs-python preinstalled, and share it with the whole team.

7. Component Implementation Plans

7.1 Code Scanner

1. Write 15 to 25 detection rules covering JavaScript, TypeScript, Python and Solidity.
2. Build three sample apps with known, documented vulnerable spots so you can state "detects N of N planted patterns".
3. Output findings in the shared schema.
4. Add a few false-positive test cases and tune the rules.

7.2 PQC Benchmark Harness

1. Wrap each algorithm behind one interface: `keygen()`, `sign(msg)`, `verify(msg, sig, pk)`.
2. Time with `time.perf_counter_ns`, warm up first, run at least 100 iterations, and report mean and standard deviation.
3. Record sizes in bytes. Export one JSON result file.

7.3 Verification Suite

1. Implement the four tamper scenarios from 4.3 for every algorithm.
2. Emit a matrix that the frontend renders directly.

7.4 MCP Servers

1. One server per domain, each exposing 2 to 4 tools with strict input schemas.
2. Allow-list paths so tools can only read the selected project folder.
3. Log every call (tool, arguments, duration) to a shared stream the UI subscribes to.

7.5 Orchestrator and Agents

1. Fixed pipeline first: Analysis, PQC Test, Verification, Decision, Report. Make it work without an LLM.
2. Add the LLM afterwards for explanations and for natural-language routing.
3. Emit events (`agent_started`, `tool_called`, `agent_finished`) over WebSocket.

7.6 Smart Contract and Chain Service

1. Contract with `anchor(bytes32)`, `getRecord(bytes32)` and an `Anchored` event.

2. Unit tests in Foundry or Hardhat.
3. Backend endpoints: `POST /anchor`, `GET /verify/{hash}`.
4. Deploy to Anvil for the demo; optionally also to Sepolia.

7.7 Voice

1. Prototype speech-to-text and text-to-speech in isolation first and measure latency.
2. Keep the command set small (about eight intents).
3. Pre-cache short spoken responses where possible to cut latency.
4. Always keep a typed-command fallback in case the room is noisy.

7.8 Frontend

1. Create the design system and page skeletons with mock data on day one.
2. Swap in real API data as each backend piece lands.
3. Add polish last: empty states, error handling, animations, onboarding.

8. Team Structure (Five Members)

Role	Owner of	Key deliverables
1. Crypto and PQC Lead	Sections 4.2, 4.3, 7.2, 7.3	liboqs Docker image, benchmark harness, verification matrix, algorithm explanations for the pitch
2. AI and Agents Lead	Sections 4.4, 7.5, Ollama	Orchestrator, agents, decision engine, LLM explanation prompts, agent event stream
3. Backend and MCP Engineer	Sections 7.4 and the API	FastAPI app, four MCP servers, shared schema, WebSocket events, report generation
4. Blockchain and Scanner Engineer	Sections 4.1, 4.5, 7.1, 7.6	Detection rules, sample apps, smart contract, anchoring and verification endpoints
5. Frontend, Voice and Demo Lead	Sections 4.6, 4.7, 7.7, 7.8	Dashboard, voice integration, latency badges, deployment, demo script and slides

Shared agreements (settle in the first hour)

- One monorepo with folders: `/frontend`, `/backend`, `/agents`, `/mcp`, `/contracts`, `/samples`, `/docs`.
- A single JSON schema file for `Finding`, `BenchmarkResult`, `VerificationResult`, `Report` and `AgentEvent`. Everyone codes against it from the start.
- Short branches, pull requests reviewed by one other member, merge to `main` at least every few hours.
- A shared Docker Compose file so anyone can run the whole stack with one command.
- A 10-minute stand-up at the start of each phase and a shared task board.

Cross-support pairs (to avoid blocking)

- Roles 1 and 3 pair on the crypto MCP server.
- Roles 2 and 5 pair on voice intent routing.
- Roles 3 and 4 pair on the anchoring endpoints.
- Role 5 tests everyone's output weekly (or hourly in a hackathon) and files bugs.

9. Roadmap

The schedule below assumes a **48-hour hackathon**. If your timeline is different, keep the order and scale the hours. Section 9.2 covers the weeks after the event.

9.1 Hackathon Timeline

Phase 0: Setup (Hours 0 to 3)

- Create the repo, Docker image with liboqs, Ollama model pulled, shared schema, task board.
- Agree on the demo storyline and which sample app is the hero.
- Ask the organizers about "Jev".

Phase 1: Foundations in parallel (Hours 3 to 14)

- Role 1: benchmark harness and verification tests working from the command line.
- Role 2: orchestrator skeleton with a fixed pipeline and mock agents.
- Role 3: FastAPI app, WebSocket event stream, first MCP server.
- Role 4: scanner rules and the first sample app; smart contract compiled and tested.
- Role 5: dashboard skeleton with mock data, design system, deployment pipeline.
- **Checkpoint at Hour 14:** each component runs standalone and outputs data in the shared schema.

Phase 2: Integration (Hours 14 to 28)

- Connect scanner, PQC tests and verification to the orchestrator through MCP.
- Decision engine produces ranked findings and recommendations.
- Report generation, hash computation and on-chain anchoring working.
- Frontend consumes real data: findings, charts, agent feed, verify page.
- **Checkpoint at Hour 28:** the full Tier 1 flow works from the browser, from scan to verified anchor. If not, cut Tier 2 items before continuing.

Phase 3: Requirement features (Hours 28 to 38)

- Local LLM explanations in the findings view.
- Voice commands with the latency badge.
- MCP call log panel and agent graph in the UI.
- Decision latency badge.
- "Local models only" demonstration and airplane-mode test.

Phase 4: Polish and hardening (Hours 38 to 44)

- Error and loading states, responsive fixes, onboarding screen.
- Rehearse the full flow repeatedly on a clean machine. Fix flaky steps.
- Record a backup demo video.
- Stretch items only if everything above is stable.

Phase 5: Pitch preparation (Hours 44 to 48)

- Final slides, README with setup instructions, architecture diagram, metrics table.
- Freeze the code. No new features.

9.2 Post-Hackathon Roadmap

Stage	Goals
Weeks 1 to 2	Clean up code, add automated tests, publish documentation, write a short blog post with the benchmark results
Weeks 3 to 6	Real repo scanning from GitHub URLs, more languages (Go, Rust), patch suggestions with diffs, hybrid signature migration guidance
Weeks 7 to 12	CI integration (GitHub Action that fails builds on quantum-vulnerable code), team accounts, report history, cost and gas modelling for several chains
Beyond	Research collaboration on how PQC signatures could be validated on-chain (precompiles, account abstraction, signature-aggregation schemes), plugin system for new algorithms and chains

10. Three-Minute Demo Script

1. **(0:00 to 0:20) Problem.** One sentence on Shor's algorithm and ECDSA. Show the "Runs locally" badge and pull the network cable or enable airplane mode.
2. **(0:20 to 0:50) Voice command.** Say "Scan the sample wallet." The agent feed lights up, MCP calls scroll by, the latency badge shows the voice response time.

3. **(0:50 to 1:20) Findings.** Show severity-ranked findings, the decision latency in milliseconds and a plain-language explanation read out loud.
4. **(1:20 to 2:00) Benchmarks.** Compare ECDSA, ML-DSA, SLH-DSA and Falcon on signature size and verify time. State the recommendation and why.
5. **(2:00 to 2:20) Verification.** Show the tamper test matrix: valid passes, modified message fails.
6. **(2:20 to 2:50) Anchoring.** Anchor the report hash, then on the Verify page upload the report (Match). Edit one character and upload again (Tampered).
7. **(2:50 to 3:00) Honest close.** State the scope limitation and the roadmap in one breath.

11. Success Metrics to Measure and Report

- Planted vulnerable patterns detected (for example, N of N) and false positives.
- Public key size, signature size, sign time and verify time for each algorithm versus ECDSA.
- Tamper-detection rate (target 100% across all scenarios).
- Decision engine latency (target under 50 ms).
- Voice round-trip latency (target under one second for simple commands).
- Time from upload to finished report.
- End-to-end flow success rate over 10 rehearsal runs.

12. Risks and Mitigations

Risk	Mitigation
liboqs fails to install on someone's machine	Shared Docker image built in Phase 0
Local LLM is slow on laptops	Use a 3B model, cache explanations for sample apps, deterministic fallback text
Voice latency too high	Keyword intent router first, pre-cached replies, typed fallback
Public testnet is slow or down	Default to local Anvil, Sepolia as optional bonus
Integration problems late in the event	Shared schema from hour one, integration checkpoint at hour 14 and 28

Risk	Mitigation
Scope creep	Tier system: nothing from Tier 2 starts until Tier 1 works end to end
Overclaiming security	Keep the scope statement in the README, UI footer and pitch
Judges question the on-chain trust model	Prepared answer: anchoring proves report integrity and timestamp, not quantum-safe authorization

13. Limitations to State Openly

- Findings come from pattern-based static analysis and will miss obfuscated or indirect usage.
- Benchmarks are measured on the demo machine and depend on hardware and the liboqs build.
- The anchoring transaction is signed with the host chain's classical keys, so the chain's own migration timeline affects long-term trust in the anchor.
- PQC signatures cannot be directly verified by most existing chains today without protocol changes or costly on-chain verification.
- Falcon is being standardized as FN-DSA, and its final form may differ from the current implementation.

14. Pre-Submission Checklist

- [ ] Organizers clarified "System One / Jev" and "Edge models" expectations
- [ ] Live URL works and a backup demo video is recorded
- [ ] README with one-command setup, architecture diagram and metrics table
- [ ] Voice and decision latency numbers measured and displayed
- [ ] Airplane-mode run completed successfully
- [ ] Verify page shows both Match and Tampered cases
- [ ] Scope disclaimer present in README, UI and slides
- [ ] All five team members can explain the full flow in one minute

15. Detailed Task Assignment by Member

This section breaks section 8's roles into concrete tasks per phase of the 48-hour plan. Hours match section 9.1. Each task has one owner. A buddy is named where a second person should review or unblock.

Member 1: Crypto and PQC Lead

Phase 0 (Hours 0 to 3)

- [ ] Build and share the Docker image with liboqs and liboqs-python preinstalled
- [ ] Confirm exact algorithm names in the installed liboqs version (ML-DSA, SLH-DSA, Falcon)

Phase 1 (Hours 3 to 14)

- [ ] Write the common interface: `keygen()`, `sign()`, `verify()` for ECDSA and each PQC algorithm
- [ ] Build the benchmark harness (warm-up, 100+ iterations, mean and standard deviation)
- [ ] Implement the four tamper tests (modified message, modified signature, wrong key, valid case)
- [ ] Output results as JSON in the shared schema

Phase 2 (Hours 14 to 28)

- [ ] Support the crypto-test and benchmark MCP servers with Member 3
- [ ] Add gas and storage estimates for larger signature sizes
- [ ] Write the recommendation rules (use case to algorithm) with Member 2

Phase 3 to 5 (Hours 28 to 48)

- [ ] Prepare the explanation of each algorithm for the pitch (one slide, plain language)
- [ ] Stretch: hybrid ECDSA plus ML-DSA demo, ML-DSA report signing
- [ ] Review all security claims in the README and slides for accuracy

Buddy: Member 3 (MCP integration)

Member 2: AI and Agents Lead

Phase 0 (Hours 0 to 3)

- [ ] Install Ollama on every machine and pull the chosen small model
- [ ] Draft the `AgentEvent` schema with Member 3

Phase 1 (Hours 3 to 14)

- [ ] Build the orchestrator skeleton with a fixed pipeline: Analysis, PQC Test, Verification, Decision, Report
- [ ] Create agent classes with mock tool responses so others can integrate early
- [ ] Implement the decision engine scoring (`risk = algorithm_weight x exposure x signature_lifetime x confidence`)
- [ ] Add a timer so decision latency is measured on every run

Phase 2 (Hours 14 to 28)

- [ ] Replace mocks with real MCP tool calls
- [ ] Emit `agent_started` , `tool_called` and `agent_finished` events over WebSocket
- [ ] Write LLM prompts that turn findings into plain-language explanations, with deterministic fallback text
- [ ] Cache explanations for the sample apps

Phase 3 (Hours 28 to 38)

- [ ] Add natural-language routing for requests the voice keyword router cannot handle (with Member 5)
- [ ] Optional: tiny scikit-learn classifier for the decision model

Phase 4 to 5 (Hours 38 to 48)

- [ ] Stress-test the pipeline with 10 repeated runs and fix flaky steps
- [ ] Prepare the architecture and agent-flow slide

Buddy: Member 5 (voice and UI events)

Member 3: Backend and MCP Engineer

Phase 0 (Hours 0 to 3)

- [ ] Create the monorepo structure and shared Docker Compose file
- [ ] Write the shared JSON schema file (`Finding` , `BenchmarkResult` , `VerificationResult` , `Report` , `AgentEvent`) and get sign-off from everyone

Phase 1 (Hours 3 to 14)

- [ ] Stand up the FastAPI app with health, scan-start and results endpoints
- [ ] Add the WebSocket event stream

- [ ] Build the first MCP server (code-analysis) with allow-listed project paths and call logging

Phase 2 (Hours 14 to 28)

- [ ] Build the crypto-test, benchmark and blockchain MCP servers (pair with Members 1 and 4)
- [ ] Implement canonical JSON report generation and hashing
- [ ] Add PDF and JSON report download endpoints
- [ ] Publish the MCP call log to the UI stream

Phase 3 to 4 (Hours 28 to 44)

- [ ] Error handling and input validation on all endpoints
- [ ] Make Docker Compose start the whole stack with one command
- [ ] Write setup instructions in the README

Phase 5 (Hours 44 to 48)

- [ ] Final backend freeze, tag the release, confirm a clean-machine install works

Buddy: Member 1 (crypto tools) and Member 4 (chain endpoints)

Member 4: Blockchain and Scanner Engineer

Phase 0 (Hours 0 to 3)

- [ ] Set up Foundry or Hardhat and a local Anvil chain
- [ ] Choose the three sample apps (token wallet, voting dApp, multisig)

Phase 1 (Hours 3 to 14)

- [ ] Write 15 to 25 detection rules for JavaScript, TypeScript, Python and Solidity
- [ ] Build the first sample app with a documented list of planted vulnerable patterns
- [ ] Write and unit-test the `ReportRegistry` contract (`anchor`, `getRecord`, `Anchored` event)

Phase 2 (Hours 14 to 28)

- [ ] Finish the other two sample apps and tune rules to reduce false positives
- [ ] Build `POST /anchor` and `GET /verify/{hash}` with Member 3
- [ ] Deploy the contract to Anvil, with optional Sepolia deployment
- [ ] Calculate the detection score (found vs planted)

Phase 3 to 4 (Hours 28 to 44)

- [ ] Build the "edit one character, show Tampered" test file for the live demo
- [ ] Stretch: patch suggestions, GitHub URL scanning

Phase 5 (Hours 44 to 48)

- [ ] Prepare the blockchain trust-model answer for judges (what anchoring does and does not prove)

Buddy: Member 3 (endpoints)

Member 5: Frontend, Voice and Demo Lead

Phase 0 (Hours 0 to 3)

- [ ] Choose the design system and colors, and set up the Next.js or React project
- [ ] Set up the task board and shared checklist

Phase 1 (Hours 3 to 14)

- [ ] Build page skeletons with mock data: New Scan, Findings, Benchmarks, Agents, Report, Verify
- [ ] Prototype speech-to-text and text-to-speech in isolation and record baseline latency
- [ ] Set up deployment for the live URL

Phase 2 (Hours 14 to 28)

- [ ] Connect each page to real API data as endpoints become available
- [ ] Build the benchmark charts (signature size, key size, sign and verify time)
- [ ] Build the live agent activity feed and MCP call log panel
- [ ] Build the Verify page with drag-and-drop

Phase 3 (Hours 28 to 38)

- [ ] Add the mic button, intent router (about eight commands) and spoken summaries
- [ ] Add latency badges for voice and decision time
- [ ] Add the "Runs locally" badge and a typed-command fallback

Phase 4 (Hours 38 to 44)

- [ ] Loading, empty and error states, responsive fixes, onboarding screen
- [ ] Record the backup demo video
- [ ] Run the full demo on a clean machine, five times

Phase 5 (Hours 44 to 48)

- [ ] Final slides, README screenshots, metrics table, demo script rehearsal with the whole team

Buddy: Member 2 (events and voice routing)

Handoffs and Dependencies

Handoff	From	To	Needed by
Shared schema signed off	Member 3	Everyone	Hour 3
liboqs Docker image	Member 1	Everyone	Hour 3
Benchmark and tamper results in schema format	Member 1	Members 2, 3, 5	Hour 14
Scanner findings in schema format	Member 4	Members 2, 3, 5	Hour 14
Orchestrator with mock agents	Member 2	Members 3, 5	Hour 10
WebSocket event stream	Member 3	Member 5	Hour 12
Contract deployed on Anvil	Member 4	Member 3	Hour 20
Report hash and anchor endpoints	Members 3 and 4	Member 5	Hour 24
Full Tier 1 flow working	All	All	Hour 28
Voice and decision latency numbers	Members 5 and 2	Slides	Hour 40

Team Rhythm

- **Stand-ups:** 10 minutes at hours 0, 14, 28 and 38. Each person answers: done, blocked, next.
- **Integration checkpoints:** hour 14 (components run standalone) and hour 28 (full flow works in the browser). If hour 28 fails, drop Tier 2 items until it passes.
- **Pull requests:** reviewed by the named buddy, merged at least every few hours.

- **Sleep schedule:** stagger rest so that at least three members are awake at all times, and plan Member 5 and Member 3 as the last two awake for final integration.
- **Pitch roles:** Member 5 presents the demo, Member 1 answers cryptography questions, Member 4 answers blockchain questions, Member 2 answers AI and agent questions, Member 3 answers architecture and MCP questions.